UNIVERSIDAD ECOTEC

Actividad Integradora 3

Grano & Co. conectada: consumo de la API REST pública de Open-Meteo con Provider

Esta actividad continúa el proyecto de las Actividades Integradoras 1 y 2 (app *Grano & Co.*, repositorio `EDWJOSH/cafeteria_app`). La nueva funcionalidad se desarrolló en la rama `feature/api-integration`.

Estudiante	Edwin Quiroz
Asignatura	Desarrollo de Aplicaciones Móviles
Docente	Gregorio Manuel Cruz Gonzabay
Fecha de entrega	7 de noviembre de 2026
Repositorio	https://github.com/EDWJOSH/cafeteria_app

Sección I. Selección y descripción de la API pública

Dato	Valor
Nombre de la API	Open-Meteo, Weather Forecast API (https://open-meteo.com)
Tipo	API REST pública y gratuita, respuestas en JSON. **No requiere registro, clave ni token** para uso no comercial.
Método	GET
Licencia de los datos	CC BY 4.0 (la app muestra la atribución “Datos del clima: Open-Meteo.com”)
Ubicación consultada	Guayaquil, Ecuador (latitud −2.19, longitud −79.89), zona horaria America/Guayaquil

Endpoint utilizado

```
https://api.open-meteo.com/v1/forecast
  ?latitude=-2.19&longitude=-79.89
  &current=temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m
  &daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max
  &timezone=America/Guayaquil&forecast_days=5
```

Parámetro	Para qué sirve
latitude / longitude	Coordenadas del local en Guayaquil
current	Variables del clima actual: temperatura, sensación térmica, humedad, código del clima y viento
daily	Variables por día: código del clima, temperatura máxima y mínima, probabilidad máxima de lluvia
timezone	Las horas y fechas llegan en la hora local de Ecuador
forecast_days	Cantidad de días del pronóstico (5)

Justificación de la elección

- **Tiene sentido para el negocio:** en una cafetería de Guayaquil el clima decide el antojo. Con calor se venden bebidas frías y con un día fresco o lluvioso, calientes. La pantalla *Clima y antojo* usa la temperatura real para **recomendar productos del menú** y permite pedirlos ahí mismo.
- **Datos en tiempo real:** la medición se actualiza cada 15 minutos, así que la información cambia de verdad cuando el usuario desliza para actualizar.
- **Cumple las restricciones del enunciado:** no hay autenticación, sesiones ni base de datos; basta una petición HTTP.
- **Es estable y está documentada.** Se probaron otras dos APIs abiertas (SampleAPIs de cafés y TheMealDB de postres), pero sus textos están en inglés y SampleAPIs es menos confiable.

Estructura del JSON consumido

Respuesta real recibida el 7 de noviembre de 2026 (recortada a dos días):

```
{
  "latitude": -2.2144113, "longitude": -79.872314,
  "timezone": "America/Guayaquil",
  "current_units": { "temperature_2m": "°C", "relative_humidity_2m": "%", "wind_speed_10m": "km/h", ... },
  "current": {
    "time": "2026-11-07T10:45",
    "temperature_2m": 29.2,
    "apparent_temperature": 36.0,
```

```
    "relative_humidity_2m": 69,
    "weather_code": 1,
    "wind_speed_10m": 1.4
  },
  "daily": {
    "time":                           ["2026-11-07", "2026-11-08", ...],
    "weather_code":                   [80, 53, ...],
    "temperature_2m_max":             [30.9, 32.9, ...],
    "temperature_2m_min":             [24.1, 24.2, ...],
    "precipitation_probability_max": [98, 63, ...]
  }
}
```

Objeto JSON	Modelo en Dart	Detalle
current	ClimaActual	Un objeto con 6 campos: cada clave del JSON pasa a un atributo con tipo (DateTime, double, int).
daily	List<PronosticoDia>	Llega como **arreglos paralelos** (una lista por campo); en fromJson se unen por posición: el día i toma time[i], weather_code[i], etc.
respuesta completa	Clima	Agrupa zona horaria, clima actual y la lista de días.
weather_code	describirClima()	Código WMO (0 despejado, 1–3 nublado, 51–57 llovizna, 80–82 chubascos...) traducido a texto e ícono.

Sección II. Arquitectura y manejo de estados

La lógica de red está **desacoplada de la interfaz** en tres piezas. La pantalla nunca hace peticiones HTTP: solo observa al **ClimaProvider** (patrón *Provider* con `ChangeNotifier`), y el provider usa el **ClimaApiService**, que es el único que conoce la URL, el paquete `http` y el formato del JSON.

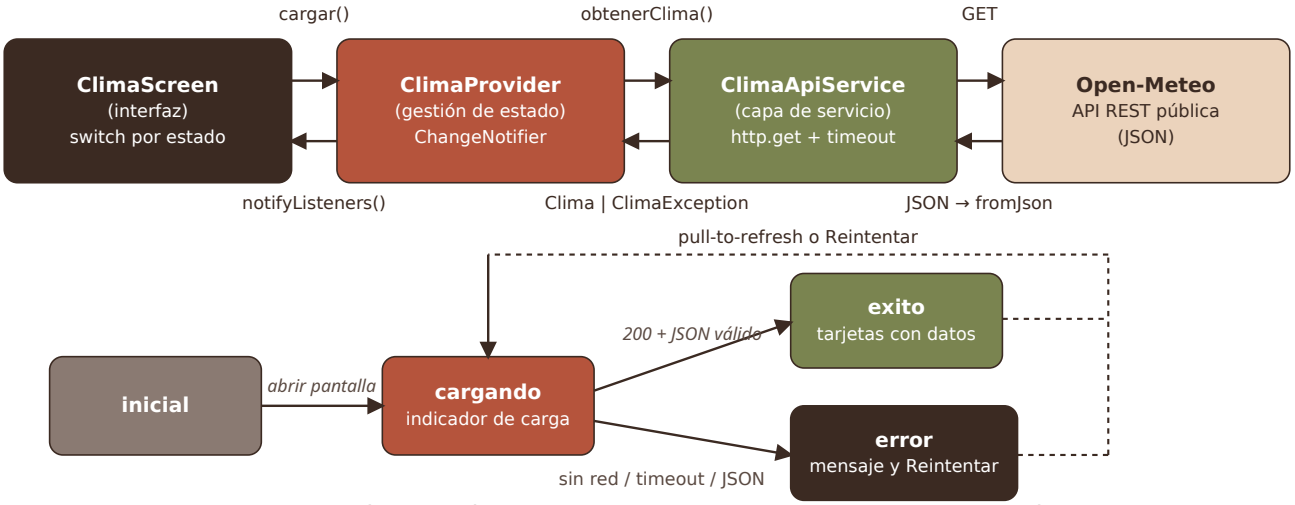


Figura 1. Arriba: capas y comunicación (la interfaz solo habla con el provider). Abajo: estados de la petición (enum EstadoClima) y sus transiciones.

Cómo se desacopló la lógica de red

- **ClimaApiService** (`lib/services/clima_api_service.dart`): arma la URL, hace el `GET` con el paquete `http`, controla el tiempo máximo y convierte cualquier fallo en una `ClimaException` con un mensaje en español. No importa nada de Flutter.
- **Modelos** (`lib/models/clima.dart`): `Clima`, `ClimaActual` y `PronosticoDia` con `fromJson` y `toJson`. El resto de la app trabaja con objetos tipados, nunca con mapas sueltos.
- **ClimaProvider** (`lib/providers/clima_provider.dart`): guarda el estado (`inicial / cargando / exito / error`), los datos, el mensaje de error y la hora de actualización. Cada cambio se anuncia con `notifyListeners()`.
- **ClimaScreen** (`lib/screens/clima_screen.dart`): con `context.watch<ClimaProvider>()` se redibuja sola y elige una vista por estado. El `RefreshIndicator` llama a `clima.cargar` al deslizar.
- El servicio se crea en `main.dart` y se **inyecta** al provider. En las pruebas se reemplaza el cliente HTTP por un `MockClient`, sin tocar la interfaz.

Sección III. Extractos de código clave

III.1 Consumo HTTP

lib/services/clima_api_service.dart (líneas 39–70) · petición GET asíncrona y manejo de errores

```
Future<Clima> obtenerClima() async {
  try {
    final respuesta = await _cliente
        .get(endpoint)
        .timeout(const Duration(seconds: 10));

    if (respuesta.statusCode != 200) {
      throw ClimaException(
        'El servicio del clima respondió con error ${respuesta.statusCode}.',
      );
    }
    final json = jsonDecode(respuesta.body) as Map<String, dynamic>;
    return Clima.fromJson(json);
  } on SocketException {
    throw const ClimaException(
      'Sin conexión a internet. Revisa tu Wi-Fi o datos móviles.',
    );
  } on http.ClientException {
    throw const ClimaException(
      'No se pudo conectar con el servicio del clima.',
    );
  } on TimeoutException {
    throw const ClimaException(
      'El servicio del clima tardó demasiado en responder.',
    );
  } on FormatException {
    throw const ClimaException('La respuesta del clima llegó dañada.');
  } on TypeError {
    // El JSON no tiene la estructura esperada (campo faltante o de otro tipo).
    throw const ClimaException('La respuesta del clima llegó dañada.');
  }
}
```

- `async / await`: la petición se hace sin bloquear la interfaz; mientras espera, Flutter sigue dibujando el indicador de carga.
- `.timeout(10 s)` evita que la app quede esperando para siempre si la red es muy lenta.
- Si el servidor no responde `200`, se lanza un error con el código HTTP.
- `jsonDecode` convierte el texto en un `Map` y `Clima.fromJson` lo convierte en objetos.
- Cada tipo de fallo tiene su mensaje: `SocketException` (sin internet), `ClientException` (no se pudo conectar), `TimeoutException` (tardó demasiado) y `FormatException` o `TypeError` (JSON dañado o con otra estructura).

III.2 Mapeo del modelo JSON

lib/models/clima.dart (líneas 69–93) · Clima.fromJson

```
/// "daily" llega como arreglos paralelos (uno por campo); aquí se unen
/// posición por posición para formar una lista de PronosticoDia.
factory Clima.fromJson(Map<String, dynamic> json) {
  final daily = json['daily'] as Map<String, dynamic>;
  final fechas = daily['time'] as List;
  final codigos = daily['weather_code'] as List;
  final maximas = daily['temperature_2m_max'] as List;
  final minimas = daily['temperature_2m_min'] as List;
  final lluvias = daily['precipitation_probability_max'] as List;

  return Clima(
    zonaHoraria: json['timezone'] as String,
    actual: ClimaActual.fromJson(json['current'] as Map<String, dynamic>),
    dias: [
      for (var i = 0; i < fechas.length; i++)
        PronosticoDia(
          fecha: DateTime.parse(fechas[i] as String),
          codigoClima: (codigos[i] as num).toInt(),
          maxima: (maximas[i] as num).toDouble(),
          minima: (minimas[i] as num).toDouble(),
```

```
          probabilidadLluvia: (lluvias[i] as num?)?.toInt() ?? 0,
        ),
    ],
  );
}
```

- Los valores numéricos se leen como `num` y se convierten con `toDouble()` / `toInt()`, porque el JSON puede traer `29` o `29.2`.
- El ciclo `for` dentro de la lista une los arreglos paralelos de `daily`: el día `i` toma la posición `i` de cada arreglo.
- `(lluvias[i] as num?)?.toInt() ?? 0` tolera un valor nulo en la probabilidad de lluvia.
- `toJson()` hace el camino inverso y reconstruye la misma estructura de la API; una prueba verifica que `fromJson(toJson(x))` devuelve los mismos datos.

III.3 Manejo de estados en la interfaz

lib/providers/clima_provider.dart (líneas 31–46) · transición de estados

```
Future<void> cargar() async {
  if (_estado == EstadoClima.cargando) return;
  _estado = EstadoClima.cargando;
  _mensajeError = null;
  notifyListeners();

  try {
    _clima = await _api.obtenerClima();
    _actualizado = DateTime.now();
    _estado = EstadoClima.exito;
  } on ClimaException catch (e) {
    _mensajeError = e.mensaje;
    _estado = EstadoClima.error;
  }
  notifyListeners();
}
```

lib/screens/clima_screen.dart (líneas 44–61) · una vista por estado + pull-to-refresh

```
Widget build(BuildContext context) {
  final clima = context.watch<ClimaProvider>();

  // Se elige qué mostrar según el estado de la petición.
  final Widget cuerpo = switch (clima.estado) {
    EstadoClima.inicial ||
    EstadoClima.cargando when clima.clima == null => const _VistaCargando(),
    EstadoClima.error => _VistaError(
      mensaje: clima.mensajeError!,
      onReintentar: clima.cargar,
    ),
    _ => _VistaDatos(clima: clima, onPedir: _pedir),
  };

  return Scaffold(
    appBar: AppBar(title: const Text('Clima y antojo')),
    body: RefreshIndicator(onRefresh: clima.cargar, child: cuerpo),
  );
```

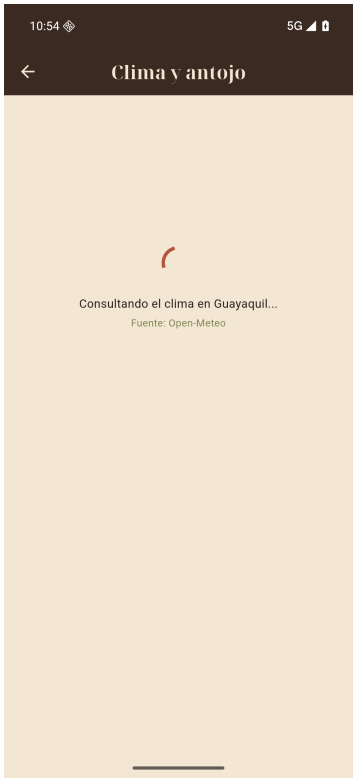
- El provider pasa a `cargando` y avisa; después termina en `exito` (con datos) o en `error` (con el mensaje). El `if` inicial evita lanzar dos peticiones a la vez si el usuario desliza varias veces.
- En la pantalla, el `switch` elige entre `_VistaCargando`, `_VistaError` y `_VistaDatos`. Si ya había datos y se está actualizando, se siguen mostrando con una barra de progreso en lugar de vaciar la pantalla.
- `RefreshIndicator(onRefresh: clima.cargar)` implementa el **pull-to-refresh**. La vista de error también es una lista desplazable, así que se puede reintentar deslizando o con el botón **Reintentar**.

Sección IV. Evidencias de pruebas (QA) y enlace al repositorio

Repositorio público: https://github.com/EDWJOSH/cafeteria_app
Rama de la funcionalidad: `feature/api-integration`, integrada en `develop`. Código nuevo en `lib/services/clima_api_service.dart`, `lib/models/clima.dart`, `lib/providers/clima_provider.dart` y `lib/screens/clima_screen.dart`.

Capturas en el emulador **Pixel_Cafe (Android 16)** consumiendo la API real. Para ver el estado de carga se limitó la red del emulador (`adb emu network speed gsm`); para el error se apagaron el Wi-Fi y los datos (`adb shell svc wifi disable`).

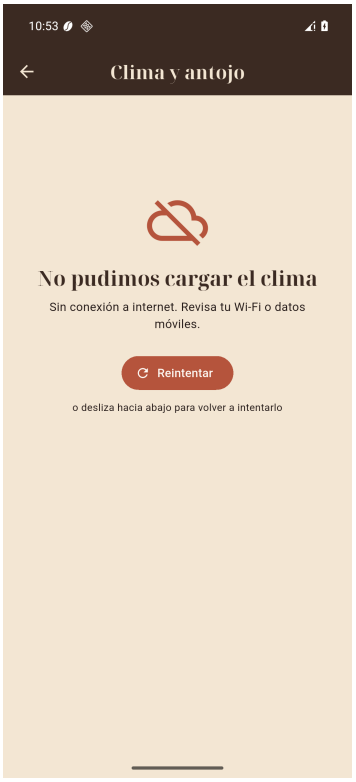
IV.1 Los tres estados



1. **Cargando**: *indicador mientras llega la respuesta.*



2. **Datos cargados**: *clima actual, recomendación y pronóstico.*



3. **Error de red**: *sin internet, con botón Reintentar.*

IV.2 Actualización manual y otros casos



4. Acceso desde la Home: tarjeta "Clima y antojo".



5. Pull-to-refresh: al deslizar hacia abajo se vuelve a consultar la API.



6. Red muy lenta: se superó el límite de 10 s y se muestra el error de tiempo.

IV.3 Pruebas automáticas

#	Grupo	Prueba (flutter test)	Resultado
1	API del clima	fromJson une los arreglos de "daily" en objetos PronosticoDia	✓ Pasó
2	API del clima	toJson devuelve la misma estructura que entrega la API	✓ Pasó
3	API del clima	HTTP 500 se convierte en ClimaException	✓ Pasó
4	API del clima	Sin red (SocketException) muestra un mensaje claro	✓ Pasó
5	API del clima	JSON con otra estructura se reporta como respuesta dañada	✓ Pasó
6	API del clima	Cargando → Éxito con datos y recomendación	✓ Pasó
7	API del clima	Error de conexión y recuperación con Reintentar	✓ Pasó
8	App (anteriores)	Con internet muestra el menú descargado de la red	✓ Pasó
9	App (anteriores)	Sin internet muestra el banner offline y el menú básico	✓ Pasó
10	App (anteriores)	Al perder la red usa la caché y al volver se recarga	✓ Pasó
11	App (anteriores)	Navega al detalle con el producto y regresa con el pedido	✓ Pasó
12	App (anteriores)	Permiso de cámara bloqueado: ofrece abrir los ajustes	✓ Pasó
13	App (anteriores)	Permiso de cámara denegado: muestra un aviso	✓ Pasó
14	App (anteriores)	Confirmar un pedido dispara la notificación del canal pedidos	✓ Pasó
15	App (anteriores)	Perder y recuperar internet notifica en el canal conectividad	✓ Pasó

Resultado: **15 de 15 pruebas pasaron** y `flutter analyze` no encontró problemas. Las pruebas usan un `MockClient` que responde con un JSON real de Open-Meteo, así que no dependen de internet. Además encontraron un error real: la temperatura de 44 px desbordaba la tarjeta en pantallas angostas, y se corrigió con `FittedBox`.

IV.4 Historial en GitHub

```
*   8d0c240 [develop] Merge feature/api-integration en develop
|\
| * 03a1294 [feature/api-integration] Docs: README con la API de Open-Meteo y cómo ejecutar la app (v2…
| * fa501cf Pruebas del consumo de API y de los estados de la pantalla
| * 5783512 Fix: la temperatura se ajusta al ancho en pantallas angostas
| * d4a8094 Pantalla Clima y antojo: tarjetas, recomendación y pull-to-refresh
| * d12e31b Estado del clima con Provider: cargando, éxito y error
| * b27402f API de clima: servicio HTTP con Open-Meteo y modelos fromJson/to…
|/
* 420e490 Docs: informe técnico PDF de la Actividad Integradora 2
```

Los 6 commits de la funcionalidad se hicieron en `feature/api-integration` y se integraron en `develop` con `merge --no-ff`. El README se actualizó con la API consumida, el endpoint, la estructura del JSON y cómo ejecutar la app.

Sección V. Conclusiones

Importancia del manejo asíncrono

- Una petición HTTP puede tardar desde milisegundos hasta varios segundos. Si se hiciera de forma síncrona, la interfaz se congelaría y Android mostraría el aviso de “la aplicación no responde”. Con `Future` y `async/await`, la app sigue respondiendo y muestra un indicador mientras espera.
- Modelar la petición como **estados explícitos** (cargando, éxito, error) hace que la interfaz siempre sepa qué mostrar y elimina las pantallas en blanco o a medio cargar.
- Hay que cuidar los casos de concurrencia: el provider ignora una segunda petición mientras la primera sigue en curso, y la pantalla comprueba `mounted` antes de usar el `context` después de un `await`.

Importancia de la gestión de errores de red

- En un móvil la red falla a menudo: túneles, Wi-Fi sin salida o datos lentos. Las pruebas lo mostraron: con la red limitada a velocidad GSM, la petición superó los 10 segundos y la app mostró un mensaje claro en lugar de quedarse cargando.
- Traducir cada excepción técnica (`SocketException`, `TimeoutException`, código HTTP, JSON inválido) a un mensaje comprensible y ofrecer **Reintentar** y **pull-to-refresh** le devuelve el control al usuario.
- No hay que confiar en que el JSON siempre venga bien: validar tipos al mapear evita que un cambio en la API cierre la app.
- Separar servicio, modelo, estado e interfaz permitió probar cada caso de error sin internet, con un `MockClient`, y agregar esta funcionalidad sin modificar las pantallas existentes.